Care Note — Round Two

Response to the sixteen clinic scenarios and the twelve-capability list. Architecture, schema and latency are unchanged and remain in `TECHNICAL_BRIEF.md`; this covers what the review changed. Per-scenario verdicts and their tests are in `SCENARIO_COVERAGE.md`.

Where we landed: 9 SURVIVES · 6 PARTIAL · 1 DOES NOT on the scenarios, from 6 · 6 · 4 in our own first assessment. Decisions D-070 to D-103, 543 backend test functions (867 parametrised cases) and 69 component tests passing.

Section 7 is the whole of what auditing ourselves turned up, grouped by cause rather than by the pass that found it. The most recent four were live while the suite was green, and two of those were misfiring on the two surfaces a patient can actually see.

Two rows moved backwards, deliberately. Scenario 3 was SURVIVES and is now PARTIAL: a patient's phone number was travelling in a query string and therefore into the access log. Scenario 2 was SURVIVES and is now PARTIAL: we had answered *where* clinic isolation is enforced and never measured what happens when that one line is wrong — the answer is every patient in both clinics, and nothing else catches it. Section 7 covers both.

`pytest tests/test_survival_scenarios.py -v` walks the sixteen scenarios one at a time, and fails if its verdicts drift from the table below.

1. What the review actually found

Our own assessment came out 6/6/4 before any code changed. The pattern in those failures was more useful than any individual fix: **what survived was structural, what failed was operational.** Access control fused into a type, redaction fused into the only code that can reach a network, highlights anchored to a version — all held. What failed was every question of the form *what happens when*: when the provider is down, when the server crashes, when a clinic needs to add a patient on a Tuesday. We had built a record system and had not built the clinic around it.

Two defaults explain most of it, and both are worth stating because they generalise beyond this build.

The stub LLM provider cannot fail. It is in-process, so it cannot time out, refuse a connection or return a 503. Every test run and demo for the entire build executed against a provider physically incapable of failing — which is why a provider outage turned out to be an unhandled 500 with a traceback. The default is still right; it makes the build runnable without an API key. But it bought that determinism by removing the failure surface from view, and nothing made the trade visible. `CARENOTE_LLM_FORCE_UNAVAILABLE` now exercises that path on every run.

The seed script stood in for features. `init_db.py` runs at Phase 1 step 1, so from the first commit every test and demo began with a populated database. *How does a patient come to exist?* never arose, because patients always already did. Anything a seed provides is a feature you have not built and will not notice missing — which is exactly scenario 1.

2. Scenario verdicts

#	Scenario	Was	Now
1	Patient with no email	PARTIAL	**SURVIVES** — staff enrolment, phone as identifier
2	Clinic isolation, one line	SURVIVES	**SURVIVES** — `AccessScope.query()`, three test files
3	Read your logs	DOES NOT	**PARTIAL** — crash path fixed; a URL leak found later (§7)
4	Prove the ordering	SURVIVES	**SURVIVES** — one exit, asserted by source scan
5	Clinic B on Monday	PARTIAL	**PARTIAL** — zero schema changes; config surface built, vocabulary still global
6	Trilingual consult	PARTIAL	**PARTIAL** — parity for en/ms; Hokkien flagged, not read
7	Allergy at minute two	DOES NOT	**DOES NOT** — batch; boundary asserted by test
8	Model hangs 45s	PARTIAL	**PARTIAL** — 8s timeout and cancel; no server-side abort
9	Provider 503 for an hour	DOES NOT	**SURVIVES** — degrades to extractive, and the card says so
10	Two people, same note	SURVIVES	**SURVIVES** — version check plus unique constraint
11	Link never received	DOES NOT	**PARTIAL** — reach modelled; no sender exists
12	Summary wrong by one dosage	PARTIAL	**SURVIVES** — correction banner and a dose gate
13	Allergy asserted vs denied	PARTIAL	**SURVIVES** — `assertion_vs_denial` at HIGH, one card per disagreement
14	A number that means nothing	SURVIVES	**SURVIVES** — floor now language-independent
15	Ranking learns from what it showed	SURVIVES	**PARTIAL** — floors, exploration and inspectability hold; bias now *measured* at 0.77 rather than argued (D-092)
16	Highlight cites edited source	SURVIVES	**SURVIVES** — now side by side, both versions named

On the twelve capabilities: **4 SURVIVES · 7 PARTIAL · 1 DOES NOT**, revised down from 6 · 5 · 1 by a second audit (§10). The single DOES NOT is streaming ASR. Full per-row detail with the tests that cover each is in `SCENARIO_COVERAGE.md`.

Four rows moved down, and the reasons differ. Two were overclaims: the correction leg of fact extraction was justified by "version history as the correction record", which is a human editing a note and not a speaker correcting themselves in a transcript; and exposure bias was marked SURVIVES on the strength of a mitigation existing, when the capability asks for an evaluation. One hid a defect (§10). The fourth, audience-appropriate output, is a capability we **declined on purpose** — no machine-written text can become patient-facing at all (D-067) — and ticking it would have claimed a feature where we had made an argument. Stated as a refusal it is the stronger answer; stated as a tick it is the thing the feedback said counts against us.

3. What changed

Failure handling (3, 8, 9). The chokepoint now translates timeouts, transport errors, 5xx and 429 into one `LLMUnavailableError`; it does not decide what to do about them. Each caller chooses, because the right answer differs by purpose: the scribe degrades to the deterministic extractive summariser it already had — labelled `offline-extractive-v1:provider-unavailable`, so a degraded note is legible as degraded — and a patient-facing generator would refuse. 4xx stays loud: a bad API key hiding behind a slightly worse summary is indistinguishable from an outage. Timeout 60s → 8s, plus a cancel button, since the transcript is stored before the model is called and abandoning loses the summary, never the consult.

Crash logging is a separate chokepoint: type, route and an eight-character reference, never `str(exc)`, which is where SQLAlchemy puts bound parameters.

Language parity (6, 14). The risk floor now works over canonical tags rather than English strings, so it inherits every language the tagger knows. It also gained negation handling, asymmetrically: a symptom is dropped only if it appears *exclusively* inside a negation, so "no chest pain Monday, chest pain today" still rates high, and a denied symptom drops to `medium` rather than to nothing. Content in an unsupported language is flagged as unread rather than silently producing nothing — abstention beats confident silence.

Denials as claims (13). Negated allergy mentions become `allergy_denial` claims instead of being discarded, and `assertion_vs_denial` fires at HIGH — not CRITICAL, because nothing dangerous has happened yet and diluting CRITICAL would break the level that means *someone is about to be given a drug they react to*.

Reach (11, 12). `unread` / `read` / `corrected`, derived from timestamps `PatientView` had recorded since D-033 and nothing had queried. `dispatched` is deliberately absent because nothing dispatches. The patient view leads with a plain-language correction banner, computed before the read marker moves.

Enrolment (1, 5). Staff can register a patient and issue a login; identifier type is explicit and a phone number is first-class. `Patient.dob` and `mrn` became nullable — both were `NOT NULL`, which was a second and quieter way the schema decided that someone known only by a phone number was not a patient.

Regeneration (capability list). Was undefined behaviour, found by probing: a new session id produced a duplicate summary, the same one crashed on a unique constraint. Now it reuses the entry and appends a version, so accepted highlights and comments survive — and refuses outright when a human has edited, because merging would mean choosing which of a clinician's sentences to keep.

Dosage (capability list, and the hint). Doses were compared against each other and never against a reference, so the build could see two entries disagree and not see 5000mg. A 17-drug adult reference now bands doses plausible / unusual / implausible; only implausible gates, and only on patient-facing writes, with the override recorded. Acknowledgement rather than refusal: a hard block teaches people to route around the check.

Tenancy (5). Scenario 5 asks config-versus-schema and the answer is now precise. *Schema: nothing* — every table carries `clinic_id`, and the seed has run two clinics since Phase 1, so a third needs no migration. *Config: everything, which was the problem* — every value a clinic might differ on was a module constant, so "Clinic B keeps records for a year" was a deploy. `ClinicConfig` (D-086) makes volume and retention per-clinic, an absent row meaning defaults. The more useful half was deciding what stays global: redaction patterns, the protected highlight classes, contradiction severities and the dosage reference are asserted *not* to be configurable, under one rule — **a clinic may change what it sees, never what it is protected from.** Anything that could be turned down until an alert stops firing sits on the left of that line. What keeps scenario 5 at PARTIAL is clinical vocabulary, still global, and the piece needing most care: additions must be additive only, or a clinic could remove `entity:allergy` and reach the safety floor sideways.

Provenance (16). Stale highlights now show the anchored text and the current text side by side, with both version numbers named.

4. What we tried that did not work

An order-of-magnitude dose threshold. Metformin 5000mg passed as merely `unusual` — the exact case the hint describes. The fix is principled rather than tuned: ranges are *single-dose*, a legitimate daily total reaches ~3× that (TDS), so beyond 3× exceeds any plausible daily total. 1500mg BD correctly stays unusual.

`@app.exception_handler(Exception)` for the log leak. Does not work. Starlette's `ServerErrorMiddleware` calls the handler and then *re-raises* so the server can log the traceback — sanitising the response and leaving the leak untouched. It had to be middleware. We only found this because the test asserted on captured log output rather than on the response body.

A blanket-denial pattern that matched too much. `denies allergy to aspirin` registered as a denial of *all* allergies. A contradiction detector that cries wolf is worse than one with gaps: the gap loses a finding, the false positive teaches people to stop reading all of them.

Streaming capture — not attempted. Retrofitting incremental extraction is tractable; `test_capture_timing.py` asserts the deterministic extractors work on two turns alone. What we could not resolve is the interruption policy. When it is acceptable to interrupt a doctor mid-consultation is a clinical workflow decision, and guessing would have produced a demo rather than a product.

5. Assumptions that no longer stand

"Logging is solved because `log_event` is content-free." Solved for content we log on purpose; silent about content logged on our behalf. One unhandled 500 put a name, an NRIC and note content in a single line. The earlier brief claimed "logs grepped, zero hits" — true of the success path, and the wrong question.

"Multilingual is fixed because the tagger handles Malay" (D-058). The fix landed at the tagger; the risk floor was a separate path making the same English-only assumption. Every Malay fixture passed because they only exercised the tagger. *When you fix an assumption at one layer, go looking for other layers that made the same one.*

"Dropping negated mentions is correct." Correct for its purpose, and it discarded the patient's denial entirely — a denial is a position, not an absence.

"The medication cue vocabulary is adequate." Built against written notes. *I will start you on amoxicillin* — how prescribing is actually said — classified as a bare dose claim and never reached the allergy comparison.

Still standing: regex redaction over NER for auditability (D-012); no HTML-escaping on write, because corrupting `dose <5mg` is worse than the XSS it prevents (D-015); full snapshots over diffs (D-006); least-privilege on staff visibility (D-004); and reporting contradictions without ever resolving them (D-068) — tested hardest, and the one we would defend most strongly.

6. Where we expect this to fail next

1. **A consult beyond English and Malay.** The unread flag means the system says so, but the content is still invisible downstream. First week.
2. **A dose the 17-drug table does not know.** No check at all, and the absence looks identical to a pass.
3. **Alert fatigue on contradictions.** Four classes now fire; nothing monitors the dismissal rate, which is the signature `NEVER_DAMPENED` exists to survive.
4. **Regeneration refusing too often.** Any human edit blocks it, including a typo fix — possibly annoying enough to teach people not to edit AI summaries.
5. **Two disagreeing clinicians in the learning loop.** Untested rather than designed; saturation bounds the damage.

Still not built, plainly: streaming ASR, acoustic diarization, any message sender, presence indicators, a real drug database, and per-clinic *clinical vocabulary* — volume and retention are configurable (§3), the watchlists are not. Scenario 7's *timing* remains DOES NOT — nothing is incremental, and `test_capture_timing.py` asserts that boundary rather than papering over it. Its *detection* half was a defect rather than a boundary, and is fixed (§7).

7. What auditing ourselves found

Everything above was written, and then the build was probed repeatedly as if by someone trying to disprove it. What follows is everything those passes found. **Almost all of it sat inside a row we had already marked SURVIVES, and none of it was caught by the tests passing at the time** — 509 of them when this started, 847 when the last four came out.

They are grouped by what they teach rather than by the pass that found them: the order is an accident of our schedule and the grouping is not. Four causes account for all of them, and two of the four are properties of how we *write tests and documents* rather than of the code.

One. The test used the shape its author had in mind. Every contradiction test used one assertion and one denial — the single shape where pairwise and grouped output are identical. A real chart re-records an allergy at every visit, so four "allergic to penicillin" entries against two denials produce eight findings that all say one thing; the card caps at five, so the copies filled the cap and an unrelated **metformin dose disagreement was evicted entirely** (D-081). The same tests never compared an entry with *itself*, which is correct for typed notes and wrong the moment an entry is a transcript: `run_scribe` writes one Entry per consult, so an allergy at minute two against a prescription at minute nineteen was undetectable at any point in its life (D-089). Abstention tests used romanised Latin script, so nobody noticed `is_unreadable` measured substance as `len(text.split()) >= 6` — a Chinese or Japanese paragraph is one token, falls under the bar, and returns False (D-090). And the first exposure-bias evaluator modelled the card its author assumed, ranking by score and taking the top N, which is not what the Glance View does (D-084) — it reported that `entity:allergy` never reaches the card, and that would have gone in this brief as a finding.

None of these were careless. Each was written from inside the assumption it needed to escape, which more of the same tests would not have fixed — **so we changed the shape of the tests rather than their number**, using property generation and schema enumeration, and that found two more. A phone number could hide behind an en-dash, because every separator class in `redaction.py` is spelled in ASCII and `\s` is Unicode-aware while the dash class is not (D-095). Clinic isolation is now enumerated from the live OpenAPI schema rather than sampled: against the exact mutation the feedback describes, hand-written RBAC tests raise 15 failures and the matrix raises 48 — both catch it, only one tells you what it exposed (D-096).

Two. The seam between two modules that are each correct alone. This is the cause we would now bet on producing the next defect, and we did not see it until the last pass. Two constants named `PATIENT_FACING_TYPES` existed in different modules holding different sets — one meaning *readable by* the patient, the other *written for* her. Both were right locally; `from ... import` had two answers and neither import site looked wrong. `delivery.py` took the first, so a note the patient typed herself was treated as clinic content that had failed to reach her, and editing it made her own view lead with *"This was updated after you last read it. If you were following the earlier version, stop and read this one"* — the loudest thing we say to a patient, fired at the one reader with no provenance rail to check it against (D-100).

Two modules extract medication-plus-dose. They shared a regex, with a comment saying so, and disagreed about which drug a dose belongs to. `contradictions` gave the first dose in a sentence to every drug in it, so *"Continue metformin 1g BD, amlodipine 5mg OD, atorvastatin 20mg ON"* produced the claim `amlodipine 1g` and a **HIGH-severity disagreement between two entries that agree**, citing a dose that does not exist for that drug. The dosage checker had the mirror fault — its window ran past the next drug name, so *"metformin and amlodipine 5mg"* read as metformin 5mg and **blocked a patient-facing write on correct prose** (D-101). One shared extractor now binds each dose to its nearest drug, bounded on both sides, which also closed a silent miss: dose-before-drug carried no dose at all, leaving a decimal slip in that phrasing invisible to the gate built to catch decimal slips.

And decay walked around our provenance mechanism. `compress` replaces an entry's content with a summary and correctly creates no version, because archival is not an authorship event — but staleness was `source_version_number != version_number`, which compression moves neither of. A cold entry reported `stale: false` while every character of its content had been replaced, so a highlight anchored to *"mild ankle swelling"* resolved to `'ing in the evenings'` and clicking it drew a box around that fragment. Precisely the "silently point at different text" outcome the mechanism exists to prevent, reached by a route it never watched (D-102). Fixing it exposed that the UI half was wrong for **edits** too, and then pushed the side-by-side banner into a case it was never written for: it rendered *"v1 → v1"*, because compression moves no version number. That last one is the cause reproducing itself one layer up.

Three. Doors nobody guards. Redaction-before-LLM is provable — no module but the wrapper may reach a model — and every leak we found was in a sink our own code does not write to. Crash logs carried SQLAlchemy bound parameters (D-071). The ASGI access log carried a phone number we had put in a URL ourselves, and it records the full request line before the application sees it, identically for requests that 404 or are refused by RBAC, so neither `log_event` nor the error middleware touches it — **the feature built to answer scenario 1 leaked through the door scenario 3 asks about** (D-083, and why scenario 3 is PARTIAL). The browser console was governed by nothing: one `console.log(entry)` left in during debugging puts a full note somewhere most deployments forward to a third-party dashboard, and the app looks identical either way.

D-095 belongs here as much as above, for its second half. `find_residual_phi` is both the fail-closed tripwire and the oracle our property tests assert against, and it shared its regexes with the redactor — so the gap was invisible twice. **A check and its own test must not share an implementation.**

Four. Legible to an auditor, not to a clinician. Section 3 above claims a degraded note is "legible as degraded". That became true of the database first; in the interface the only trace was a 10px grey monospace model string in the provenance footer, so during an hour-long outage the card read as an ordinary AI summary (D-082). No error boundary existed, so any component throwing unmounted the whole tree — and in a clinical record a blank page is indistinguishable from data loss, so the recovery a clinician reaches for is retyping a note they never lost. Sessions expire at 60 minutes with no refresh (D-016), which fires on a real clinic laptop most afternoons, and surfaced as "Token expired" beside a chart that had silently stopped working. And losing the network — not an edge case for a bedside PWA — reached the clinician as Chrome's own "Failed to fetch", silent on the only question that matters mid-consult: did the note save.

The one number we had a mechanism for and no measurement of. Exposure bias is now measured: displacement 0.15, **exposure concentration 0.77**, blind-tag rate 0.16, zero protected classes displaced (D-092). Ten of thirteen visible slots go to tags this clinic has already given feedback on. We are not claiming that is a good number; we have nothing to compare it to. We are claiming it moves when the system changes, which is what an argument cannot do. Relatedly, `NEVER_DAMPENED` had been our answer to "what stops ranking burying an allergy" and it floors the wrong quantity — surfacing is a top-N cut, so other tags rising displaces an allergy with its own weight untouched, and one dismissal removed it from the card permanently. Protected classes now bypass ranking entirely (D-084): learning orders the protected set, it no longer decides membership.

What held, stated because "we checked" is a different claim from "we assumed". Patient-role isolation across nine endpoints probed with a patient token — zero fragments of staff notes, clinician sections or raw AI summaries in any response body. The service worker is network-only for `/api`, so nothing is written to disk on a shared device. Four property suites: content round-trip through the real API, analyser totality and determinism over the full unicode range, revision-history invariants over generated edit/revert sequences, and log hygiene greping every logger at DEBUG for synthetic identifiers. **Every property was mutation-checked**, because a test that has never failed is a test whose teeth are unmeasured. And the RBAC dependency itself: there is no unscoped query path to reach for, and `Version` — the one model without a `clinic_id` — is only ever reached through a clinic-scoped `Entry`.

Enforcement that is strong but singular. Access control is fused into a type and redaction into the only module that can reach a network. Both are impossible to forget; neither is defended in depth. Dropping the clinic predicate from `AccessScope.query` exposes every patient in both clinics and nothing else catches it (D-085). We had been treating *unforgettable* as if it were *redundant*, and the scenario asks about the second.

What we did not fix, and why. Keyword tagging has no notion of negation, so "no anaphylaxis" emits `symptom:anaphylaxis` — a known limitation with a stated reason. What was *not* documented is its interaction with the protection list: that tag is in `NEVER_DAMPENED`, so a note ruling anaphylaxis out produces a tag that can never be learned down. The clinician dismisses it and the floor discards the dismissal — **the anti-alert-fatigue mechanism manufacturing a permanent false positive.** Separately, space-separated identifiers (`900101 01 5432`, what a patient reading an IC aloud transcribes to) survive redaction; widening the pattern puts every run of grouped clinical digits at risk of becoming `[ID_1]`, trading a narrow privacy gap for a broad accuracy one, which is exactly what the hint warned against. Both are pinned by tests that assert current behaviour and fail the day anyone changes it.

What none of this covered. The voice-capture and ASR pipeline, the concurrency paths beyond their existing tests, and the learning loop past its accumulator were read but never probed with the adversarial input that produced everything above; D-103 records that boundary rather than leaving the absence of a finding to read as a clean bill. On the evidence — the seam cause accounting for the most recent and least anticipated defects — that is where we would look next, and specifically at what `capture` assumes about `scribe`.

One prediction, recorded because it did not help us. An earlier draft of this section ended by saying we expected a fourth overclaim to exist and named the rows we would probe first: **12 and 15**. Two of the last four defects are in scenario 12. We wrote down where we thought the build was weakest and then shipped two more rounds of work before going to look.